Случайное блуждание (Random Walk)

Случайное блуждание (Random Walk)

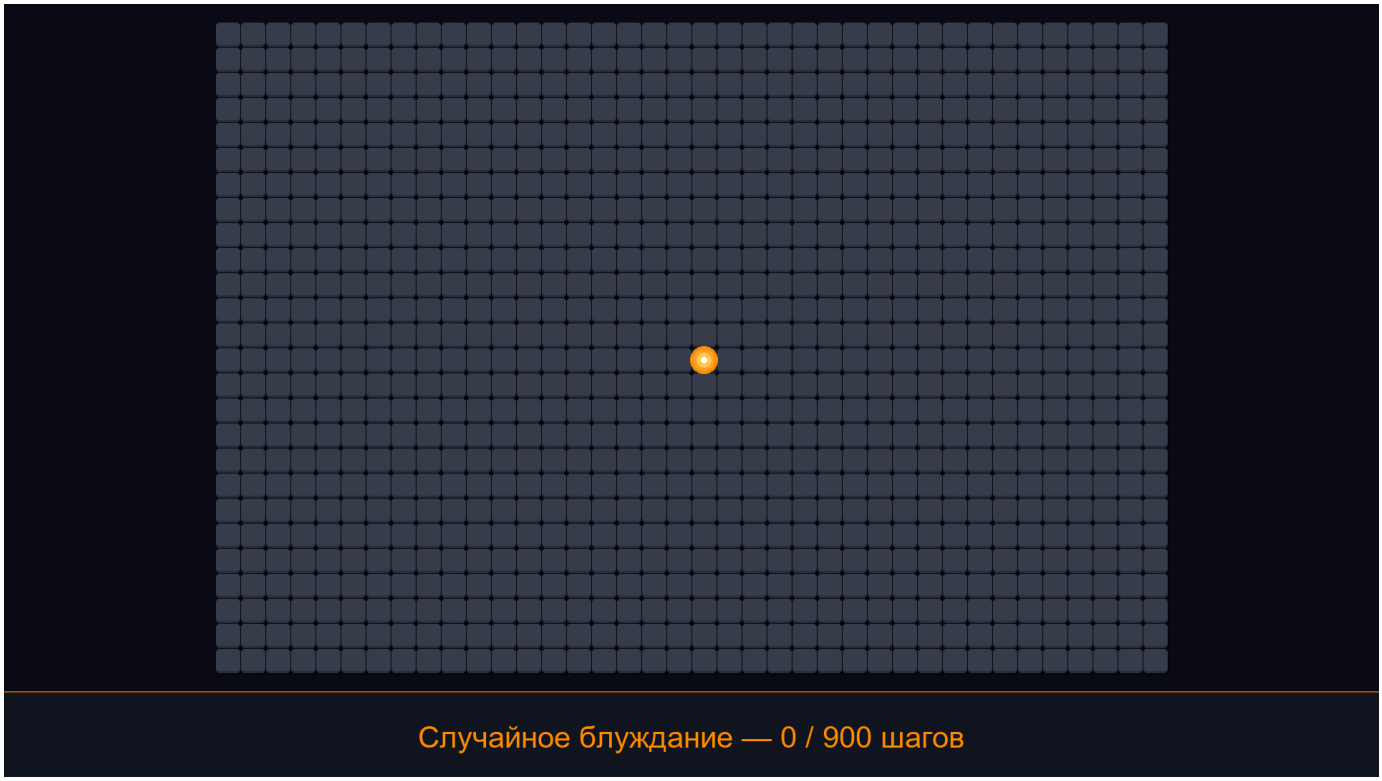
Идея

Ну, самый наивный способ сгенерировать уровень - раскидать стены и пол рандомом по каждой ячейке. Проблема очевидна: получится несвязная каша, через которую нельзя пройти. А что, если мы будем не разбрасывать, а "вырезать" проходы?

Представим, что весь уровень - это матрица, полностью заполненная стенами. Темнота. И у нас есть "**строитель**" - агент, который стоит где-то в центре этой матрицы. На каждом шаге он случайно выбирает направление - вверх, вниз, влево, вправо - перемещается туда и превращает ячейку под собой в пол. Т.е. он "прорезает" проходы в стенах.

После тысячи таких шагов у нас получается извилистая система тоннелей и небольших открытых пространств. Никакой симметрии, никаких прямых углов - все органическое, хаотичное, напоминающее естественные пещеры.

Почему этот алгоритм стоит рассмотреть первым? Потому что он элементарный. Буквально 15-20 строк кода, и у вас уже есть работающий генератор уровней.



Реализация

Алгоритм укладывается в один цикл. Разберём его ядро - шаг блуждания.

Шаг блуждания

На каждой итерации строитель выбирает случайное направление и, если новая позиция в пределах матрицы, перемещается туда и отмечает ячейку как пол. Проверка `1 <= nx < width - 1` оставляет по краям стены толщиной в один тайл - уровень "закрыт" по периметру.

Python C#

```python
import random

directions = [(0, -1), (0, 1), (-1, 0), (1, 0)]

for _ in range(steps):
    dx, dy = random.choice(directions)
    nx, ny = x + dx, y + dy
    # leave a 1-tile wall border
    if 1 <= nx < width - 1 and 1 <= ny < height - 1:
        x, y = nx, ny
        grid[y][x] = 0
```

Результат

Что мы получаем? Связную систему проходов. "Связную" - потому что строитель не телепортируется, он идёт шаг за шагом, т.е. каждая прорезанная ячейка соединена с предыдущей. Если мы поставим старт в точку, откуда начал строитель, а выход - в точку, где он закончил, то путь между ними гарантированно существует.

Форма уровня зависит от параметров:

- **Мало шагов** (скажем, 200 на сетке 40x30): узкие, короткие тоннели. Большая часть уровня - стены.
- **Много шагов** (5000+): строитель проходит по одним и тем же местам повторно, "расширяя" пещеры. Получаются открытые пространства.
- **Размер сетки**: чем больше сетка при том же количестве шагов, тем более разреженным будет результат.

Попробуйте поменять параметры: `random_walk(40, 30, 300)`, `random_walk(40, 30, 3000)`, `random_walk(80, 60, 5000)`. Посмотрите, как меняется результат.

Вариации

Ок. Базовый алгоритм прост, но у него есть очевидные ограничения. Вот несколько модификаций, которые могут сделать результат интереснее.

Несколько строителей

Вместо одного "строителя" запускаем несколько. Каждый начинает из своей точки (или из одной общей) и прорезает свой путь. Результат: более равномерное покрытие матрицы, несколько "зон" пещеры.

Но тут появляется нюанс: раз строители начинают из разных точек, нет гарантии, что их пещеры соединятся. Т.е. мы можем получить несколько изолированных "островков". Тут пригодится валидация с заливкой, чтобы проверить, связан ли уровень, и, если нет, прорезать коридор между регионами или перегенерировать.

Целевое покрытие

Вместо фиксированного количества шагов мы задаём процент ячеек, которые должны стать полом. Допустим, мы хотим, чтобы 40% уровня было проходимым. Строитель ходит, пока не "прорежет" нужное количество ячеек.

Это даёт более предсказуемый результат - мы точно знаем, какая часть уровня будет проходимой. Но при высоком проценте заполнения строитель будет долго блуждать по уже прорезанным участкам, прежде чем найдёт новую стену. Т.е. алгоритм может работать медленно.

Направленное блуждание

А что, если строитель не совсем случаен? Мы можем добавить ему "предпочтение" - например, с вероятностью 50% он продолжает идти в том же направлении, что и на предыдущем шаге. Это создаёт длинные прямые коридоры вместо компактных клубков.

Или наоборот - мы можем заставить строителя чаще менять направление, что даёт более плотные, компактные пещеры.

Модификация: сохраняйте текущее направление между шагами. В начале каждого шага с вероятностью `1 - keep_prob` выбирайте новое случайное направление, иначе - продолжайте в том же.

Все эти вариации можно комбинировать: несколько строителей с направленным блужданием и целевым покрытием. Экспериментируйте, смотрите, что лучше подходит для вашей игры.

Направленное блуждание - по сути, простейшая цепь Маркова: вероятность следующего шага зависит от текущего направления. Если вероятность "продолжить прямо" = 50%, а "повернуть" = по 16.7% на каждое из трёх направлений - это марковская матрица переходов 4×4.

Плюсы и минусы

- **Простота**: самый простой алгоритм генерации. Легко реализовать, легко понять, легко модифицировать.
- **Органичность**: результат не выглядит "искусственным" - никаких ровных линий и прямоугольников.
- **Связность**: базовый вариант (один строитель) гарантирует, что все прорезанные ячейки связаны.

Но:

- **Отсутствие структуры**: нет комнат, нет коридоров в привычном смысле. Если вашей игре нужны чёткие комнаты с дверями - этот алгоритм не подойдёт.
- **Непредсказуемость формы**: может получиться длинная узкая змейка, а может - бесформенное пятно. Контроля над формой мало.
- **Неравномерное покрытие**: строитель часто возвращается на уже пройденные ячейки, поэтому некоторые области матрицы могут остаться нетронутыми.

Что дальше?

- Клеточные автоматы - тоже создают пещеры, но более гладкие и естественные. Хорошо дополняют случайное блуждание.
- BSP-генерация - если вам нужны именно комнаты с коридорами, а не пещеры.
- Шум Перлина - для ландшафта и открытых миров.
- L-системы - не уровни, а растения и фракталы - другая ветвь процедурной генерации.
- Wave Function Collapse - генерация из правил соседства.

Powered by Wiki.js